

# Image Restoration with Autoencoders, Hard Routing and a Soft Mixture of Experts, and Style-Conditioned Face-to-Sketch Generation with a Conditional GAN

Abubakar [full name]

Generative AI, Assignment 1 — Roll No. [roll number]

National University of Computer and Emerging Sciences (FAST-NUCES)

Code: [https://github.com/Abubakar7-dotcom/Gen\\_ai\\_assignment1](https://github.com/Abubakar7-dotcom/Gen_ai_assignment1)

**Abstract**—We build and deploy four related generative systems. On the Oxford-IIIT Pet dataset, images corrupted at runtime by salt-and-pepper noise, Gaussian blur or rectangular occlusion are restored by (1) a single universal denoising autoencoder, (2) a corruption classifier that hard-routes each image to one of three specialist autoencoders or an identity bypass, and (3) a jointly fine-tuned soft mixture of experts (MoE) whose gate blends the identity branch and the three experts. On FS2K, (4) a pix2pix-style conditional GAN with a learned style embedding in both generator and discriminator turns a face photograph into a sketch of one of three styles. Every task is tuned with Optuna, tracked in Weights & Biases, exported to ONNX (max. PyTorch-ONNX Runtime difference  $6.6 \times 10^{-6}$ ) and served by one React + FastAPI application started with a single `docker compose` command. The classifier reaches 99.6% accuracy (macro-F1 0.993) and the MoE gate routes 99.3% of test images to the correct branch. Restoration quality is limited to about 23 dB by the compressed  $8 \times 8$  latent code: removal of salt-and-pepper noise and occlusions improves PSNR by 6.5–7 dB, but blurred and clean inputs lose detail. An ablation shows that a single narrow skip connection raises validation PSNR from 22.4 to 27.3 dB, identifying the spatial bottleneck as the main limit.

**Index Terms**—denoising autoencoder, mixture of experts, conditional GAN, pix2pix, Optuna, ONNX, image restoration

## I. INTRODUCTION

Real images are degraded in different ways, and a restoration system usually does not know which degradation it faces. This assignment studies three ways to handle several corruptions with the same budget of autoencoders: one shared model for everything (Task 1), an explicit classifier that picks a specialist (Task 2), and a differentiable gate that blends specialists (Task 3). Task 4 moves from restoration to paired image-to-image generation and asks for a single generator that produces three distinct sketch styles. The four models are integrated into one containerised browser application so that an evaluator can corrupt or upload unseen images and inspect the routing decisions, weights and timings.

Our contributions are: a single corruption implementation shared by training, evaluation and the application (Section III); Optuna studies for all four tasks with documented search spaces (Section IV); per-condition and per-severity results with

failure analysis for each task (Sections V–VIII); two ablations that explain the main weakness of the restoration models (Section V-A); and a deployment that is tested end to end from a fresh clone (Section IX).

## II. RELATED WORK

**Denoising autoencoders.** Vincent *et al.* [1], [2] train an autoencoder to reconstruct a clean input from a corrupted one, which forces the code to capture structure rather than copy pixels. Restoration networks such as RED-Net [3] and DnCNN [4] reach much higher fidelity by adding symmetric skip connections or by predicting the residual, i.e. by letting the input bypass the bottleneck. The assignment forbids unrestricted skips so that the latent code stays a genuine compression; Section V-A measures what this costs.

**Loss functions.** SSIM [5] compares local luminance, contrast and structure. Zhao *et al.* [6] showed that an  $\ell_1$  term combined with (MS-)SSIM gives better restorations than  $\ell_2$ , which over-smooths; this motivates the  $\alpha \ell_1 + (1 - \alpha)(1 - \text{SSIM})$  loss used here.

**Mixtures of experts.** Jacobs *et al.* [7] introduced a gating network that softly assigns inputs to experts. Sparsely-gated MoEs [8] and Switch Transformers [9] add auxiliary load-balancing losses because gates tend to collapse onto a few experts; our balance term follows the same idea. Hard routing (Task 2) corresponds to a top-1 gate trained separately from the experts.

**Conditional GANs.** GANs [10] learn a generator through an adversarial discriminator; conditional GANs [11] feed side information to both networks. Pix2pix [12] combines a U-Net generator [13], a PatchGAN discriminator that judges local patches, and an  $\ell_1$  term with  $\lambda = 100$ . FS2K [15] provides photo-sketch pairs in three styles and benchmarks this kind of model; earlier face-sketch work used exemplar-based synthesis [16].

**Tools.** Optuna [18] implements the Tree-structured Parzen Estimator (TPE) [19] and median pruning; Weights & Biases [23] tracks runs; ONNX Runtime [24] runs exported models without PyTorch.

### III. DATA AND CORRUPTION PIPELINE

**Oxford-IIIT Pet** [17]. The official `trainval` list (3,680 images) is split 80/20 with seed 42 into 2,944 training and 736 validation images; the 3,669 official test images are used only for the final evaluation. Images are converted to RGB, resized to  $128 \times 128$  (area interpolation) and cached once as a `uint8` array; corrupted copies are never stored. The same split is used for Tasks 1–3.

**Runtime corruption.** One NumPy/OpenCV module implements the four conditions (Table I) and is imported by training, manifest generation, evaluation and the FastAPI backend, so the application behaves exactly like the reported experiments. During training every loaded image draws a new condition and severity. A batch sampler gives every batch exactly  $B/4$  images of each condition, which satisfies the balanced-batch requirement deterministically. Occlusion coverage is measured as the area of the *union* of the rectangles and resampled until it lies in range, because summing rectangle areas over-counts overlaps.

**Deterministic evaluation.** A validation manifest stores one condition per validation image (184 per condition) and a test manifest stores, for each of the 3,669 test images, the clean version plus every corruption at three fixed severities (36,690 items). Each entry records type, severity, parameters (probability, kernel and  $\sigma$ , rectangle coordinates) and seed. Fig. 1 shows samples.

TABLE I  
CORRUPTION CONFIGURATION (TRAINING RANGES AND FIXED TEST SEVERITIES)

| Condition     | Training (runtime, uniform)                                   | Test low / medium / high                       |
|---------------|---------------------------------------------------------------|------------------------------------------------|
| Clean         | unchanged                                                     | —                                              |
| Salt & pepper | $p \sim U(0.02, 0.15)$ ; pixel set to black or white w.p. 0.5 | $p = 0.03 / 0.08 / 0.15$                       |
| Gaussian blur | $k \in \{3, 5, 7\}$ , $\sigma \sim U(0.5, 2.5)$               | $(k, \sigma) = (3, 0.7) / (5, 1.5) / (7, 2.5)$ |
| Occlusion     | 1–3 black rectangles, union area 10–35%, random positions     | 1 / 2 / 3 rectangles, $\approx 10 / 20 / 35\%$ |

**FS2K** [15]. We use the official split: 1,058 training and 1,046 test pairs. A stratified 15% validation split (seed 42) gives 899 training and 159 validation pairs (styles 1/2/3: 303/298/298 train, 54/52/53 val). A pairing script verified all 2,104 photo–sketch pairs (0 problems); RGBA and grayscale files are converted to RGB. Augmentation is a random horizontal flip and a random resized crop (resize to 136, crop 128) whose parameters are drawn once and applied to *both* photo and sketch, preserving pixel correspondence.

### IV. EXPERIMENTAL SETUP

**Hardware and training.** All models were trained on an NVIDIA RTX 4050 Laptop GPU (6GB) with PyTorch 2.11 and `bfloat16` autocast. Autoencoders, classifier and MoE use AdamW [20] with a cosine schedule (with 100 linear warm-up steps for the autoencoders and the classifier, and gradient clipping at norm 1 for the MoE); the GAN uses Adam [21] ( $\beta_1 = 0.5$ ) with a constant learning rate for

half of the epochs followed by linear decay, as in `pix2pix`. Two engineering choices were measured before the long runs: `cuda.benchmark` was disabled because its auto-tuning added about 70s per run without a steady-state speed-up, and data loading runs in the main process because Windows worker processes each re-import CUDA PyTorch (1–4 GB of committed memory), which exhausted the paging file and hung two parallel runs.

**Hyperparameter optimisation.** Every task uses an Optuna study with the TPE sampler (seed 42) and a MedianPruner (3 start-up trials), stored in SQLite under `optuna_studies/`. Trials are short (4 epochs on a 50% training subset for Tasks 1–2; 1 warm-up + 3 joint epochs for Task 3; 10 epochs for Task 4) and the best configuration is then retrained for the full schedule. Table II lists the search spaces and the selected values. The restoration objective is

$$J = \frac{1}{2} \min(\text{PSNR}, 40) / 40 + \frac{1}{2} \text{SSIM}, \quad (1)$$

so both terms lie in  $[0, 1]$  and a perfect identity output (infinite PSNR) cannot dominate. The classifier is selected by validation macro-F1, and the GAN by  $\frac{1}{2}(1 - \ell_1) + \frac{1}{2} \text{SSIM}$  on the validation sketches. The budget (10–15 trials) is small; this trade-off is discussed in Section X.

TABLE II  
OPTUNA SEARCH SPACES AND SELECTED VALUES

| Parameter                                                                                   | Range                                     | Selected                    |
|---------------------------------------------------------------------------------------------|-------------------------------------------|-----------------------------|
| <i>T1 universal DAE: 15 trials (10 complete, 5 pruned), best #9, <math>J = 0.468</math></i> |                                           |                             |
| learning rate                                                                               | $[3 \cdot 10^{-4}, 3 \cdot 10^{-3}] \log$ | $4.61 \cdot 10^{-4}$        |
| batch size                                                                                  | {32, 64, 128}                             | 32                          |
| latent channels (code = $c \times 8 \times 8$ )                                             | {8, 16, 32, 64}                           | 8 (512 values)              |
| encoder base channels                                                                       | {24, 32, 48}                              | 48                          |
| dropout                                                                                     | [0, 0.3]                                  | 0.002                       |
| $\alpha$ ( $\ell_1$ weight)                                                                 | [0.5, 0.95]                               | 0.730                       |
| <i>T2 classifier: 15 trials (8 complete, 7 pruned), best #14, macro-F1 0.977</i>            |                                           |                             |
| learning rate                                                                               | $[3 \cdot 10^{-4}, 3 \cdot 10^{-3}] \log$ | $1.00 \cdot 10^{-3}$        |
| batch size                                                                                  | {32, 64, 128}                             | 32                          |
| channels                                                                                    | small/medium/wide                         | {32, 64, 128, 256}          |
| dropout                                                                                     | [0, 0.5]                                  | 0.249                       |
| weight decay                                                                                | $[10^{-6}, 10^{-2}] \log$                 | $2.05 \cdot 10^{-4}$        |
| <i>T2 specialists (shared search on blur): 12 trials (8/4), best #9 same space as T1</i>    |                                           |                             |
| same space as T1                                                                            |                                           | same as T1                  |
| <i>T3 soft MoE: 10 trials (9 complete, 1 pruned), best #7, <math>J = 0.865</math></i>       |                                           |                             |
| joint learning rate                                                                         | $[10^{-5}, 3 \cdot 10^{-4}] \log$         | $2.82 \cdot 10^{-5}$        |
| temperature $\tau$                                                                          | [0.5, 2.0]                                | 0.647                       |
| $\lambda_1$ ( $\lambda_s = 1 - \lambda_1$ )                                                 | [0.5, 0.95]                               | 0.864                       |
| $\lambda_c$ (gate CE)                                                                       | [0.01, 0.5] $\log$                        | 0.145                       |
| $\lambda_b$ (balance)                                                                       | $[10^{-3}, 0.1] \log$                     | 0.0076                      |
| <i>T4 cGAN: 10 trials (8 complete, 2 pruned), best #8, objective 0.687</i>                  |                                           |                             |
| G / D learning rate                                                                         | $[5 \cdot 10^{-5}, 5 \cdot 10^{-4}] \log$ | $3.21 / 3.94 \cdot 10^{-4}$ |
| batch size                                                                                  | {8, 16, 32}                               | 8                           |
| generator base channels                                                                     | {32, 48, 64}                              | 64                          |
| dropout                                                                                     | {0, 0.25, 0.5}                            | 0.25                        |
| style-embedding dim                                                                         | {8, 16, 32}                               | 32                          |
| $\lambda_{L1}$                                                                              | [10, 200] $\log$                          | 168.6                       |

**Tracking.** Each final run logs hyperparameters, every loss term, validation metrics, sample grids from a fixed validation batch and the best checkpoint (as a model artifact) to W&B (<https://wandb.ai/abubaknaveed-national-university-of-computer-and-emergi/>)

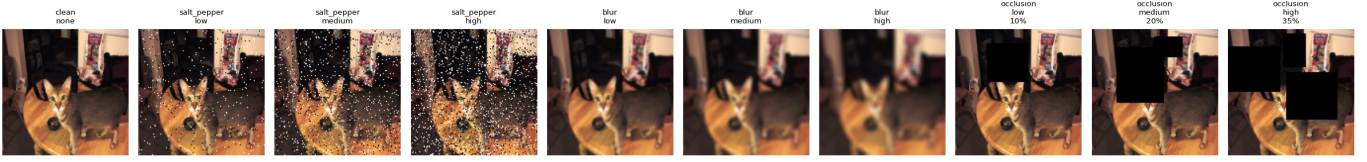


Fig. 1. Corruption sanity grid produced by the shared corruption module (clean, salt-and-pepper, blur and occlusion at several settings). It was inspected before training to confirm value ranges and that occlusion masks respect the union-area limits.

genai-a1). The training curves in this report are rebuilt from the local W&B run files.

## V. TASK 1: UNIVERSAL DENOISING AUTOENCODER

**Architecture** (Fig. 4). A stem and four stride-2 stages reduce  $128 \rightarrow 8$  pixels while widening channels to  $b \cdot (1, 2, 4, 8)$  with  $b = 48$ ; each stage has two  $3 \times 3$  convolutions with batch norm and GELU. A  $1 \times 1$  convolution maps to the latent code of  $c$  channels at  $8 \times 8$ . The decoder mirrors the encoder with bilinear upsampling followed by convolutions, which avoids the checkerboard artefacts of transposed convolutions [22], and ends with a sigmoid. With the selected  $c = 8$  the code holds 512 values for a 49,152-value input ( $96\times$  compression), so the model cannot copy its input. It has 5.39M parameters. The loss is  $\mathcal{L} = \alpha \ell_1 + (1 - \alpha)(1 - \text{SSIM})$  with  $\alpha = 0.730$  chosen by Optuna (the brief suggests 0.8 as a starting point).

**Training.** The selected configuration was trained for 40 epochs (22 min). Fig. 2 shows a smoothly decreasing training loss. Validation PSNR rises for clean, salt-and-pepper and blurred inputs together and levels off near 23 dB after about 25 epochs, while occluded inputs stay about 2.5 dB lower and fluctuate more, because inventing the content behind a mask is the hardest of the four problems.

**Results.** Table III reports the untouched test set. The model removes salt-and-pepper noise (+6.5 dB, SSIM  $0.38 \rightarrow 0.65$ ) and fills occlusions (+6.8 dB), and the improvement grows with severity because the corrupted input becomes worse while the output stays near 23 dB. For blurred and clean inputs the output is *worse* than the input: the output quality is almost the same for every input (22.9–23.0 dB), which is the signature of an information bottleneck. Fine textures that survive mild blur do not fit through a 512-value code, so the network returns a smooth reconstruction of the main shapes (Fig. 3). The error maps concentrate on fur texture, edges and, for occlusions, the masked region.

**Failure cases** (Fig. 5). The four lowest-SSIM test outputs (SSIM 0.19–0.21) are all high-severity occlusions that hide most of the animal in front of a textured background (grass, flowers). The code keeps the coarse layout, but the decoder fills the masks with a blurred average of the surrounding colours and also smooths away the grass texture, so the error is spread over the whole image. It cannot hallucinate a head or fur pattern, because  $\ell_1$  and SSIM both reward the conditional mean rather than sharp but uncertain detail. The first case is different: the photograph has a genuinely black background, and the model “fills” it like an occlusion, which produces the

TABLE III  
TASK 1 TEST RESULTS (PSNR IN DB / SSIM). CLEAN INPUT IS IDENTICAL TO THE TARGET.

| Condition             | Severity | PSNR in  | PSNR out | SSIM in | SSIM out |
|-----------------------|----------|----------|----------|---------|----------|
| Clean                 | —        | $\infty$ | 22.89    | 1.000   | 0.653    |
| Salt & pepper         | low      | 20.15    | 22.92    | 0.601   | 0.652    |
|                       | medium   | 15.87    | 22.95    | 0.337   | 0.651    |
|                       | high     | 13.14    | 22.92    | 0.201   | 0.647    |
| Blur                  | low      | 32.15    | 22.99    | 0.937   | 0.654    |
|                       | medium   | 26.89    | 23.12    | 0.804   | 0.649    |
|                       | high     | 24.53    | 23.00    | 0.695   | 0.632    |
| Occlusion             | low      | 16.69    | 21.73    | 0.862   | 0.627    |
|                       | medium   | 13.37    | 20.55    | 0.732   | 0.598    |
|                       | high     | 10.80    | 18.98    | 0.549   | 0.550    |
| All 36,690 test items |          | 27.36*   | 22.20    | 0.672   | 0.631    |

\*Includes clean inputs, whose PSNR is capped at 100 dB.

bright error region — the same black-region confusion that misleads the Task 2 classifier.

### A. Ablations: bottleneck size and one limited skip

To find the cause of the 23 dB ceiling we retrained Task 1 twice with everything else unchanged (Table IV). Increasing the code from 8 to 64 channels (4,096 values) improves validation PSNR by only 0.8 dB. Adding *one* narrow skip — the  $32 \times 32$  encoder feature squeezed to 8 channels by a  $1 \times 1$  convolution (8,192 values, still  $6\times$  smaller than the input) and joined to the decoder at the same resolution — raises PSNR by 4.9 dB and SSIM from 0.64 to 0.84. Blur now ends above its input (28.8 dB versus about 28 dB) instead of 5 dB below it. The limit is therefore the  $8 \times 8$  spatial resolution of the code, not its channel count, which is consistent with the success of skip connections in restoration networks [3]. Fig. 8 shows much sharper outputs; occlusion holes, however, are only blurred over, and a thin bright border appears on salt-and-pepper inputs because noisy edge pixels leak through the skip. The submitted models keep `skip=false` to respect the “genuine bottleneck” requirement without exception; adopting the skip would require retraining Tasks 1–3.

TABLE IV  
TASK 1 ABLATIONS (VALIDATION, LAST OF 40 EPOCHS, PSNR DB)

| Model                         | $J$ best     | PSNR         | SSIM         | S&P          | Blur         | Occl.        |
|-------------------------------|--------------|--------------|--------------|--------------|--------------|--------------|
| Selected (code 8 ch, no skip) | 0.600        | 22.39        | 0.639        | 22.96        | 23.22        | 20.48        |
| Code 64 ch, no skip           | 0.630        | 23.15        | 0.681        | 23.84        | 24.11        | 20.82        |
| Code 8 ch + one limited skip  | <b>0.763</b> | <b>27.28</b> | <b>0.844</b> | <b>28.89</b> | <b>28.83</b> | <b>22.13</b> |

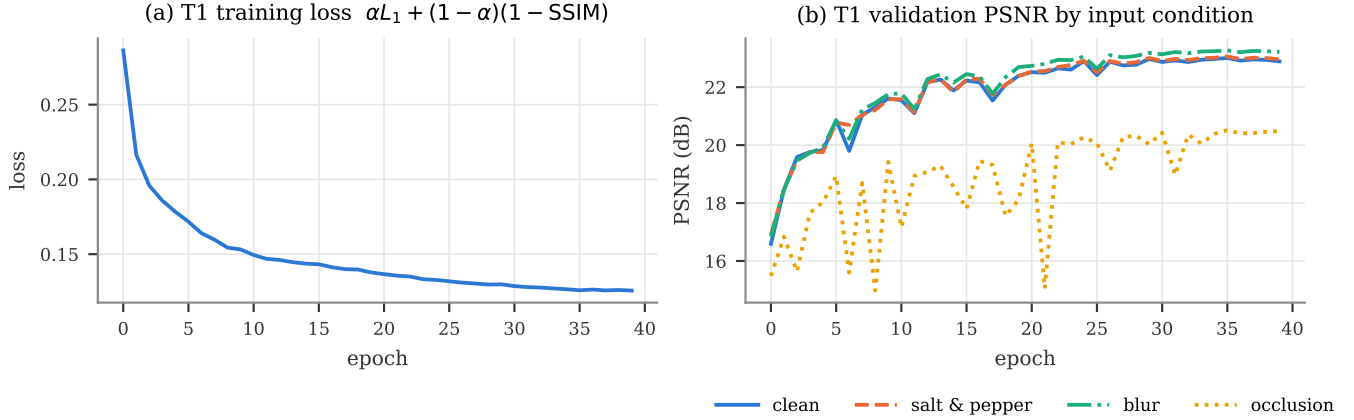


Fig. 2. Task 1 training loss and validation PSNR per input condition (validation manifest, 736 images).

**Optuna analysis** (Fig. 9, top left). In the 4-epoch trials, dropout explains most of the variance of the objective (importance 0.51; any noticeable dropout slows early learning), followed by  $\alpha$  (0.29) and the learning rate (0.19). Code size, batch size and width had importance below 0.01: within four epochs they made no measurable difference, so the study’s choice of the smallest code was effectively arbitrary. The 40-epoch ablation shows that the code size *does* matter for a full run (+0.8 dB with 64 channels), i.e. short trials hid an effect that appears only later in training. A longer trial budget or a multi-fidelity schedule would reduce this bias.

## VI. TASK 2: CLASSIFIER AND HARD-ROUTED SPECIALISTS

**Classifier.** Four convolutional stages (32–256 channels, two  $3 \times 3$  conv + BN + ReLU each, then  $2 \times 2$  max pooling), global average pooling, dropout and a linear layer (1.17M parameters) predict  $p = [p_{\text{clean}}, p_{\text{salt}}, p_{\text{blur}}, p_{\text{occ}}]$ . It is trained with cross-entropy on balanced batches for 25 epochs and selected by validation macro-F1.

**Specialists.** Three autoencoders with the Task 1 architecture are trained independently, each only on its own corruption, for 30 epochs. A shared Optuna search (12 trials, run on the blur specialist as a proxy) chose the architecture. Because the search space, sampler seed and the first 10 random start-up trials of TPE were identical to Task 1, it selected the same configuration. Inference follows the brief:  $r = \arg \max_k p_k$ ; clean images use the identity bypass and are never processed by an expert (Fig. 6a).

**Classifier results.** The test accuracy is 99.59% with macro precision / recall / F1 of 0.991 / 0.995 / 0.993 (Table V, Fig. 10). Salt-and-pepper and blur are recognised perfectly. The only systematic confusion is between *clean* and *occlusion*: 117 low-severity occlusions are called clean and 30 clean images are called blur (23) or occlusion (7). Fig. 7 shows that validation accuracy reaches 99% within six epochs.

**Oracle versus predicted routing.** Table VI compares all restoration systems per condition. Oracle and predicted routing

TABLE V  
TASK 2 CLASSIFIER ON THE TEST MANIFEST (36,690 ITEMS)

| Class         | Precision | Recall | F1    | Support        |
|---------------|-----------|--------|-------|----------------|
| Clean         | 0.969     | 0.992  | 0.980 | 3,669          |
| Salt & pepper | 1.000     | 1.000  | 1.000 | 11,007         |
| Blur          | 0.998     | 1.000  | 0.999 | 11,007         |
| Occlusion     | 0.999     | 0.989  | 0.994 | 11,007         |
| Macro average | 0.991     | 0.995  | 0.993 | accuracy 0.996 |

are almost identical (they differ only for the 0.8% of clean images and 3.2% of low occlusions that are misrouted), so the operational system is limited by the specialists, not by the classifier. Surprisingly, the specialists are *not* better than the universal model on their own corruption (e.g. blur 22.77 versus 23.04 dB): they share the same 512-value bottleneck, which removes the potential advantage of specialisation. The real gain of hard routing is the identity bypass: clean images are returned untouched (SSIM 1.0) instead of being degraded to 22.9 dB by the universal model.

TABLE VI  
TEST PSNR (DB) / SSIM PER CONDITION FOR ALL RESTORATION SYSTEMS

| Condition     | Input                       | T1 universal        | T2 oracle       | T2 predicted | T3 soft MoE         |
|---------------|-----------------------------|---------------------|-----------------|--------------|---------------------|
| Clean         | $\infty$ /1.000             | 22.89/0.653         | $\infty$ /1.000 | 99.4*/0.997  | 93.3*/0.996         |
| Salt & pepper | 16.39/0.380                 | <b>22.93</b> /0.650 | 22.66/0.652     | 22.66/0.652  | 22.66/0.652         |
| Blur          | <b>27.86</b> / <b>0.812</b> | 23.04/0.645         | 22.77/0.647     | 22.77/0.647  | 22.78/0.647         |
| Occlusion     | 13.62/ <b>0.714</b>         | 20.42/0.592         | 20.51/0.595     | 20.52/0.598  | <b>20.57</b> /0.605 |

\*Mean over items whose MSE is capped at  $10^{-10}$  (100 dB) when the identity branch returns the input.

**Misrouting failures** (Fig. 13). The four restorations hurt most by a classifier error are all *clean* photographs that contain genuine black regions — letterbox bars or a black background. The classifier has learned “large black rectangle  $\Rightarrow$  occlusion”, which is exactly the training signal, so these images are sent to the occlusion expert, which blurs the whole image (SSIM drops from 1.0 to 0.28–0.48). Conversely, small low-severity masks that hide little structure are called clean and

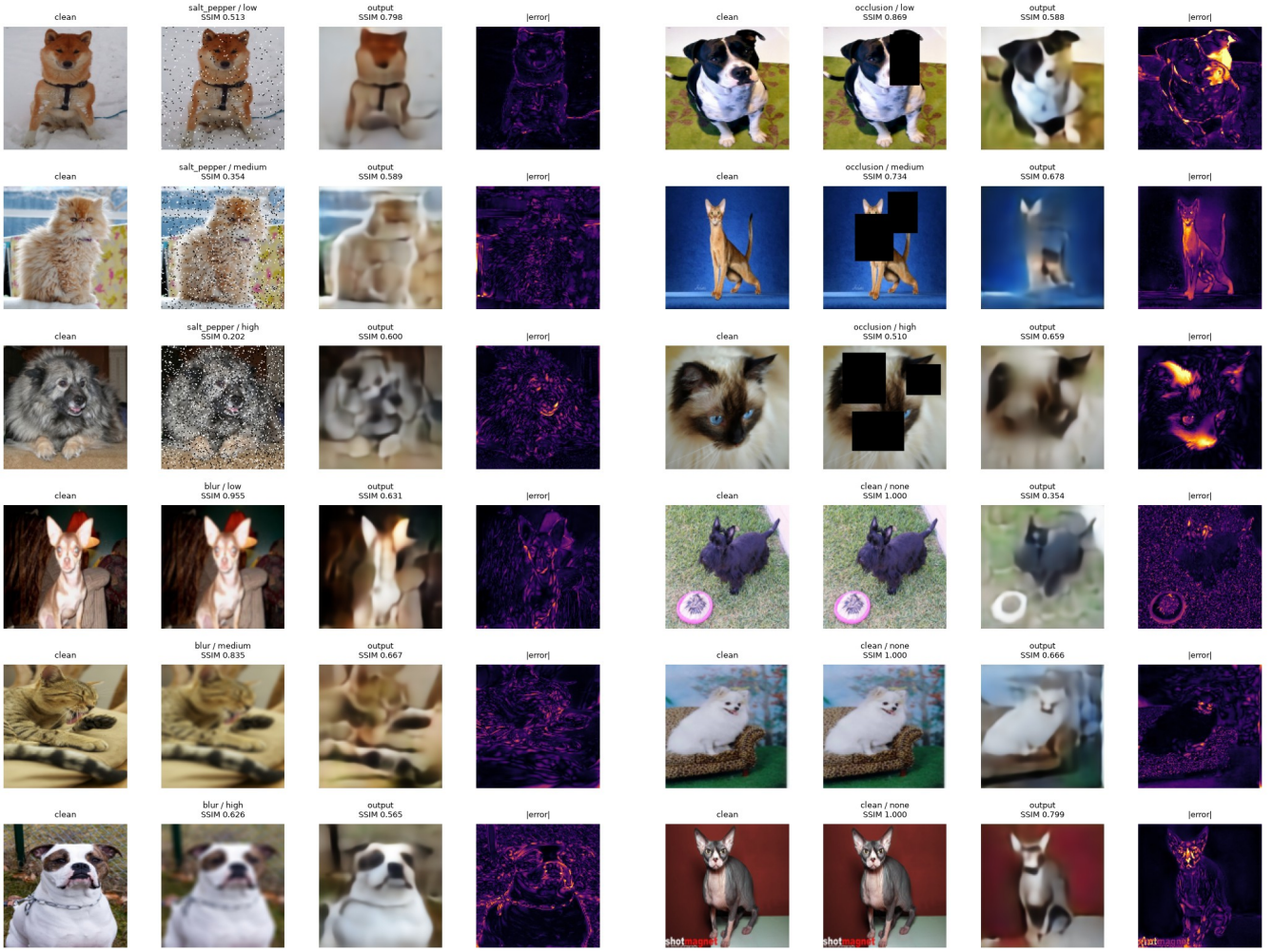


Fig. 3. Task 1: twelve test examples (one per corruption and severity plus three clean images). Columns per example: clean target, corrupted input, reconstruction, absolute error map (bright = large error).

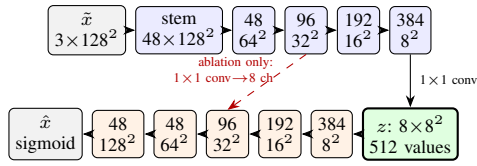


Fig. 4. Task 1 denoising autoencoder (channels / spatial size). Encoder (blue) halves the resolution four times; the latent code (green) is the only path to the decoder (orange). The dashed limited skip exists only in the ablation of Section V-A; the submitted model has no skip.

pass unrestored. Both errors come from the definition of the corruption rather than from under-training; a remedy is to add letterboxed clean images to training or to let a soft gate hedge between branches, which Task 3 does.

## VII. TASK 3: SOFT MIXTURE OF EXPERTS

**Model.** The gate  $G$  is a copy of the Task 2 classifier and the experts are copies of the three specialists, plus an identity branch (Fig. 6b):  $w = \text{softmax}(G(\tilde{x})/\tau)$  and

$\hat{x} = w_0 \tilde{x} + w_1 A_{\text{salt}}(\tilde{x}) + w_2 A_{\text{blur}}(\tilde{x}) + w_3 A_{\text{occ}}(\tilde{x})$  (17.3M parameters). Training starts with a 3-epoch warm-up in which the experts are frozen and only the gate learns (learning rate  $10^{-3}$ ), followed by 12 epochs of joint fine-tuning of all parameters with the smaller Optuna-selected rate  $2.8 \cdot 10^{-5}$ . The loss is

$$\mathcal{L} = \lambda_1 \ell_1 + \lambda_s (1 - \text{SSIM}) + \lambda_c \text{CE}(G(\tilde{x}), y) + \lambda_b \sum_{k=0}^3 \left( \bar{w}_k - \frac{1}{4} \right)^2, \quad (2)$$

where  $\bar{w}_k$  is the mean weight of branch  $k$  in a (balanced) batch. The balance term follows the auxiliary losses of sparse MoEs [8], [9]; because every batch contains exactly one quarter of each condition, its optimum coincides with correct routing, so it penalises collapse without fighting the CE term. Optuna trials whose largest mean branch weight exceeded 0.9 on validation were treated as routing collapse and pruned.

**Training behaviour** (Fig. 11). The validation objective rises during warm-up ( $0.832 \rightarrow 0.856 \rightarrow 0.863$ ) and then stays flat between 0.860 and 0.862 for all 12 joint epochs. The

Task 1: lowest-SSIM failure cases

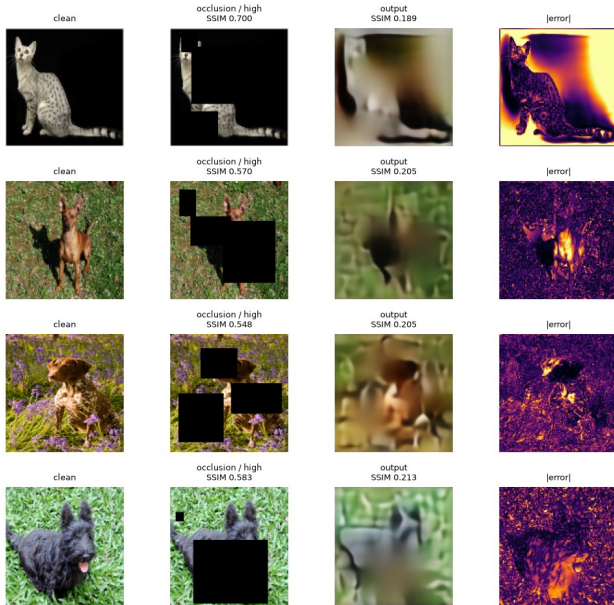


Fig. 5. Task 1 failure cases (lowest output SSIM, distinct images).

checkpoint with the best validation objective is therefore the *end of warm-up*; in the delivered MoE the experts equal the Task 2 specialists and only the gate was trained. We kept this selection rule rather than overriding it, and also release the final joint checkpoint. Joint fine-tuning cannot help much here: the experts are capped by their bottleneck (Section V-A) and a quarter of the objective comes from clean images, where the identity branch already reaches the 40 dB cap in (1).

**Routing analysis.** The gate assigns 99.3% of test images to the correct branch. Fig. 16 reports the mean weights for every true condition and severity: salt-and-pepper and blur receive a weight of 1.00 on their expert at every severity; clean images give 0.98 to the identity branch; and low-severity occlusion is the only cell with a noticeable split (0.93 occlusion expert, 0.07 identity), the same ambiguity that the classifier showed. The share of images whose largest weight goes to each branch is 0.104 / 0.300 / 0.301 / 0.295 (identity / salt / blur / occlusion), close to the manifest proportions 0.1 / 0.3 / 0.3 / 0.3, so no expert is inactive and none dominates unrelated inputs.

Fig. 12 contrasts dominant and distributed routing. The three lowest-entropy cases are salt-and-pepper images routed with weight 1.00. The three highest-entropy cases are low-severity occlusions, e.g. weights (0.16, 0, 0.61, 0.23): the small mask makes the image look almost clean or slightly blurred, and the soft gate hedges. Hedging is what Task 3 was designed for: on occlusion the MoE has the best SSIM of all systems (0.605, Table VI), and for low occlusions it improves SSIM from 0.636 (hard routing) to 0.657 because uncertain images are no longer sent wholesale to the wrong branch. A weighted average of a sharp and a blurred reconstruction can, however, also produce ghosting, as in the distributed example with

weight 0.61 on the blur expert.

#### VIII. TASK 4: STYLE-CONDITIONED FACE-TO-SKETCH cGAN

**Architecture** (Fig. 17). The generator is a pix2pix U-Net [12], [13] with seven stride-2 downsamplings ( $128 \rightarrow 1$ ), instance normalisation [14], dropout in the three innermost decoder blocks and skip connections at every scale (42.1M parameters; skips are allowed here because the input and output share structure). The style  $s \in \{1, 2, 3\}$  is a learned `nn.Embedding(3, 32)` that is injected twice into the generator — tiled as 32 extra input channels and concatenated again at the  $1 \times 1$  bottleneck — and once into the discriminator. The discriminator is a PatchGAN (2.8M parameters) on the concatenation of photo, sketch and tiled style embedding, producing  $14 \times 14$  real/fake logits, i.e. it judges whether each local patch is a plausible sketch *of that style* for *that photo*. Losses use binary cross-entropy with logits:  $\mathcal{L}_D = \text{BCE}(D(x, y, s), 1) + \text{BCE}(D(x, G(x, s), s), 0)$  and  $\mathcal{L}_G = \text{BCE}(D(x, G(x, s), s), 1) + \lambda_{L1} \|y - G(x, s)\|_1$  with  $\lambda_{L1} = 168.6$  selected by Optuna (pix2pix uses 100).

**Training** (Fig. 14). The selected configuration (batch 8) was trained for 150 epochs (33 min). The discriminator losses stay around 0.3–0.4 and decrease slowly as the learning rate decays, while the generator’s adversarial loss stays around 2.3–2.5: neither network wins, so training is stable. The training  $\ell_1$  keeps falling, but the validation  $\ell_1$  is flat after about 20 epochs and the best validation score occurs at epoch 47; afterwards the generator mainly fits training-set stroke detail, which is expected with 899 training pairs. The checkpoint selected on validation is therefore from epoch 47.

**Results.** On the 1,046 official test pairs the generator reaches  $\ell_1 = 0.103$  and SSIM 0.481 (Table VII). Styles 1 and 3, which are line drawings and soft shaded sketches with large white areas, are reproduced well (SSIM 0.52 and 0.61). Style 2, drawn with dense dark hatching, is the hardest (SSIM 0.40): even a plausible sketch places its individual strokes differently from the artist, and pixel-aligned metrics punish every misplaced stroke. Fig. 15 shows that the embedding has a real effect: the same photograph rendered as another style changes line weight and shading accordingly, so the condition is not ignored.

TABLE VII  
TASK 4 TEST RESULTS PER STYLE (1,046 OFFICIAL TEST PAIRS)

| Style   | $\ell_1$ ( $\downarrow$ ) | SSIM ( $\uparrow$ ) |
|---------|---------------------------|---------------------|
| Style 1 | 0.076                     | 0.523               |
| Style 2 | 0.152                     | 0.397               |
| Style 3 | 0.064                     | 0.613               |
| All     | 0.103                     | 0.481               |

**Failure cases** (Fig. 20). The lowest-SSIM outputs are all style-2 portraits with long hair. The generated sketches are recognisable and in the right style, but hair strands and hatching do not align with the reference, which yields SSIM below 0.21. Background clutter can also be drawn as strokes.

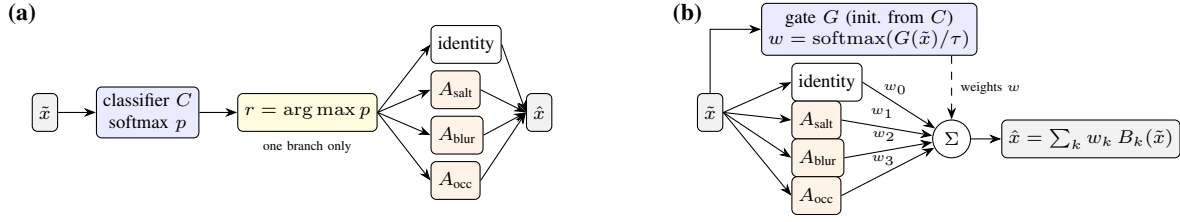


Fig. 6. (a) Task 2 hard routing: the classifier picks exactly one branch; clean inputs bypass restoration. (b) Task 3 soft MoE: every branch runs and the gate’s continuous weights blend the identity branch and the three experts; the whole pipeline is differentiable and exported as one ONNX graph that outputs the image and the weights.

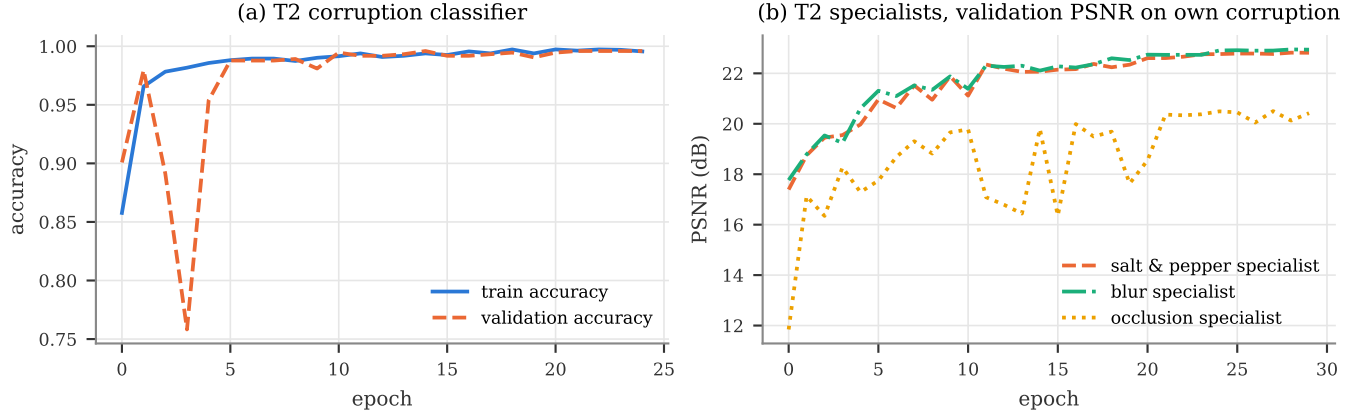


Fig. 7. Task 2 training curves: classifier accuracy (validation shows one transient dip at epoch 3, early in training when the learning rate is at its peak) and validation PSNR of each specialist on its own corruption.

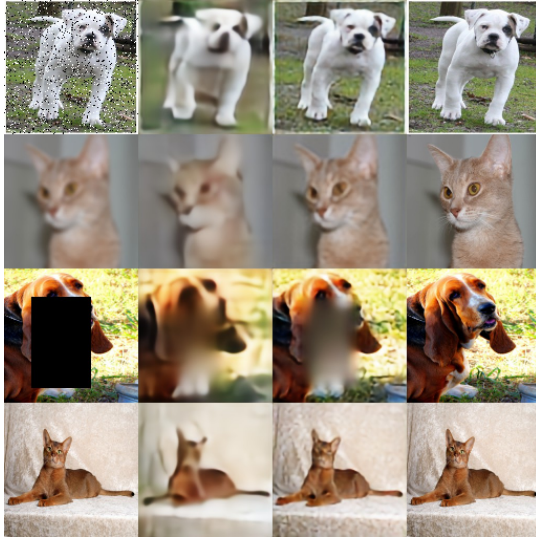


Fig. 8. Ablation on validation images. Columns: corrupted input, selected model (no skip), model with one limited skip, clean target. Rows: salt-and-pepper, blur, occlusion, clean.

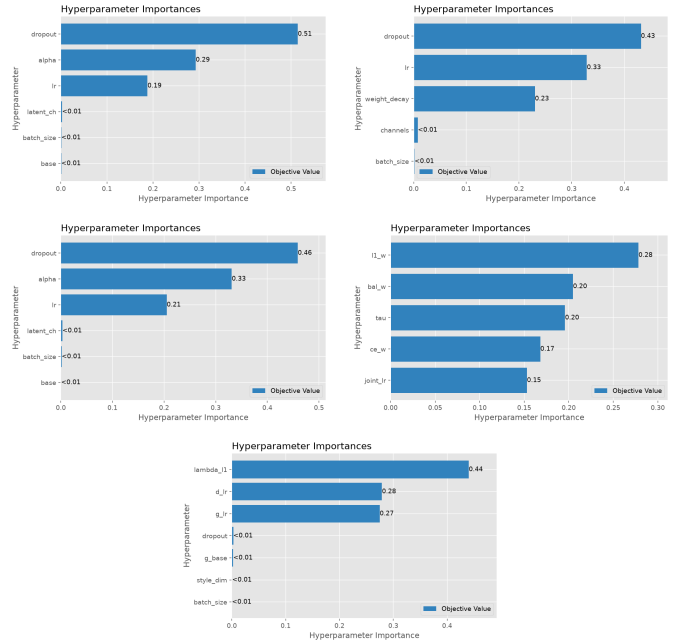


Fig. 9. Optuna hyperparameter importance (fANOVA) of the five studies: Task 1 DAE, classifier, shared specialist search, soft MoE and cGAN (left to right, top to bottom). Optimisation histories are in `results/*/optuna/`.

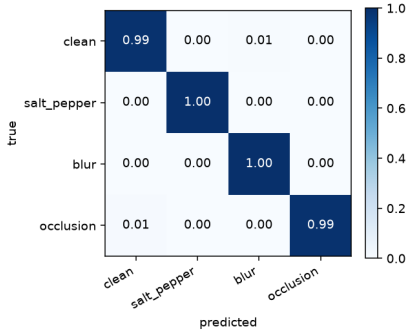


Fig. 10. Normalised confusion matrix of the corruption classifier (rows: true class).

A data issue also affects the per-style numbers: the official test annotation labels 179 sketches from the `photo3` folder as style 1, although the sketches we inspected resemble the pencil style labelled style 3 elsewhere. We kept the official labels as required, and note that per-style test numbers may be slightly mixed.

**Optuna across tasks** (Fig. 9). For the classifier, dropout (0.43), learning rate (0.33) and weight decay (0.23) explain almost all variance, while the channel configuration and batch size do not matter in short trials. The specialist search repeats the Task 1 pattern (dropout,  $\alpha$ , learning rate). For the MoE the importance is spread over all five parameters (reconstruction weight 0.28, balance weight and  $\tau$  0.20 each, classification weight 0.17, joint learning rate 0.15): the gate, initialised from an accurate classifier, is already close to optimal, so each parameter changes the objective only slightly. For the GAN,  $\lambda_{L1}$  is the most important parameter (0.44), followed by the two learning rates (0.28 and 0.27); style-embedding size, width, dropout and batch size had little effect in 10-epoch trials, and the search moved  $\lambda_{L1}$  above the usual 100 [12].

## IX. APPLICATION AND DEPLOYMENT

**Design.** The interface was specified in Google Stitch (prompt in `docs/stitch/stitch_prompt.md`): one page with a status pill, four tabs named exactly as in the brief, an input card on the left and image panels, statistics and task-specific cards on the right (Fig. 18). It was implemented in React with Tailwind CSS (Fig. 19).

**Architecture** (Fig. 21). Docker Compose starts two containers. The *frontend* container builds the React app with Node 22 and serves it with nginx, which also proxies `/api/` to the backend. The *backend* container runs FastAPI with ONNX Runtime on the CPU. Its endpoints are GET `/health`, `/samples`, `/corruptions`, and POST `/corrupt`, `/restore/universal`, `/restore/hard`, `/restore/moe` and `/sketch`. Uploads are validated (image MIME type, decodable, at most 10 MB; errors 415/413/422), converted to RGB and resized to  $128 \times 128$  exactly as in training; runtime corruption calls the same module used for training. Responses contain the images (base64 PNG),

inference time, the classifier probabilities, predicted class and selected expert (Task 2) or the four MoE weights with their ranking (Task 3). ONNX sessions are created lazily, and a missing model returns HTTP 503 with an explanation instead of crashing.

**One-command start.** The evaluator runs `git clone`, then `docker compose up --build`, and opens `http://localhost:8080`. On its first start the backend downloads the seven ONNX files (330 MB) from the GitHub Release, verifies their SHA-256 checksums and writes them atomically to a mounted folder, so later starts reuse them; download scripts for Windows and Linux are provided as an alternative. A GitHub Actions workflow reproduces the evaluator’s procedure on a clean Linux machine on every change: fresh clone, build, start, an end-to-end test of all four workspaces and the upload validation through nginx, a restart, and the same test again. It passed, as did the same test run locally against the real nginx.

**ONNX export.** All seven inference models were exported (opset 17) and compared with PyTorch on the same inputs (Table VIII); the largest absolute difference is  $6.6 \times 10^{-6}$ , far below the  $10^{-4}$  tolerance. The MoE is exported as one graph that returns both the restored image and the four weights. On a laptop CPU a request takes roughly 30–200 ms after the first (model-loading) call.

TABLE VIII  
ONNX MODELS AND PYTORCH–ONNX RUNTIME PARITY

| Model                        | Params  | Size (MB) | Max abs. diff.            |
|------------------------------|---------|-----------|---------------------------|
| T1 universal DAE             | 5.39 M  | 20.6      | $2.4 \times 10^{-6}$      |
| T2 classifier                | 1.17 M  | 4.5       | $1.2 \times 10^{-7}$      |
| T2 specialists (each)        | 5.39 M  | 20.6      | $\leq 4.9 \times 10^{-6}$ |
| T3 soft MoE (whole pipeline) | 17.34 M | 66.2      | $2.6 \times 10^{-6}$      |
| T4 generator                 | 42.12 M | 160.7     | $6.6 \times 10^{-6}$      |

## X. LIMITATIONS

(1) The restoration models are limited by the 512-value spatial bottleneck: they improve heavily corrupted inputs but degrade mild blur and clean images, and specialists do not outperform the universal model. The limited-skip ablation shows a large gain but was not adopted. (2) The Optuna budgets (10–15 short trials on half of the data) bias the search towards fast-learning configurations, as the bottleneck choice shows; a multi-fidelity method (e.g. Hyperband) or longer trials would be more reliable. (3) Joint MoE fine-tuning did not improve validation, so the delivered MoE uses the warm-up checkpoint. (4) Pixel metrics underrate stylised sketches, especially style 2; a perceptual or FID-style evaluation would complement them, and FS2K test style labels are partly inconsistent. (5) All results come from single training runs (seed 42) without confidence intervals. (6) Inputs are processed at  $128 \times 128$ , so large uploaded photos lose detail before restoration.

## XI. CONCLUSION

We implemented, tuned, evaluated and deployed four generative models in one application. Explicit routing is almost

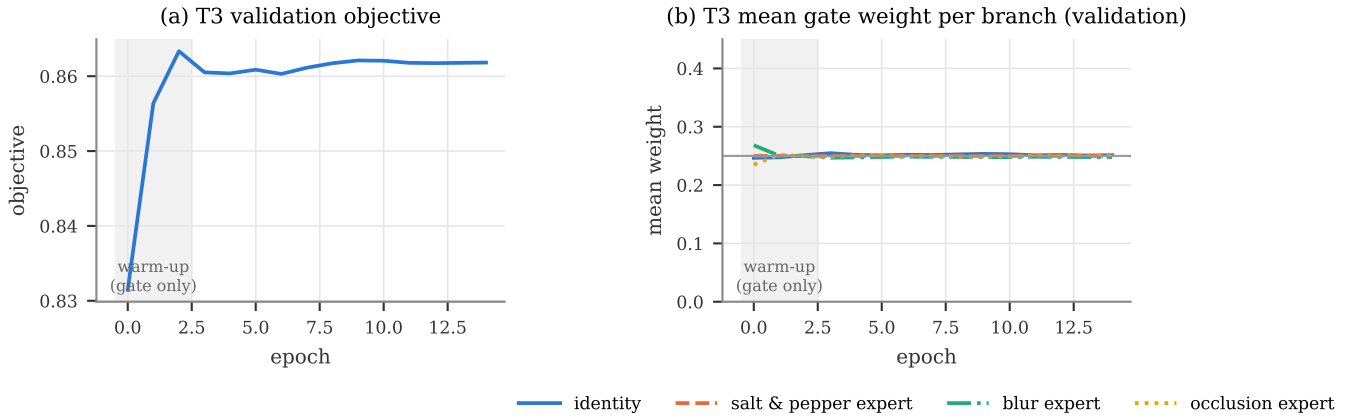


Fig. 11. Task 3 validation objective and mean gate weight per branch. The shaded epochs are the warm-up (experts frozen). The weights stay at  $\approx 0.25$  each on the balanced validation set: no branch collapses or dies.

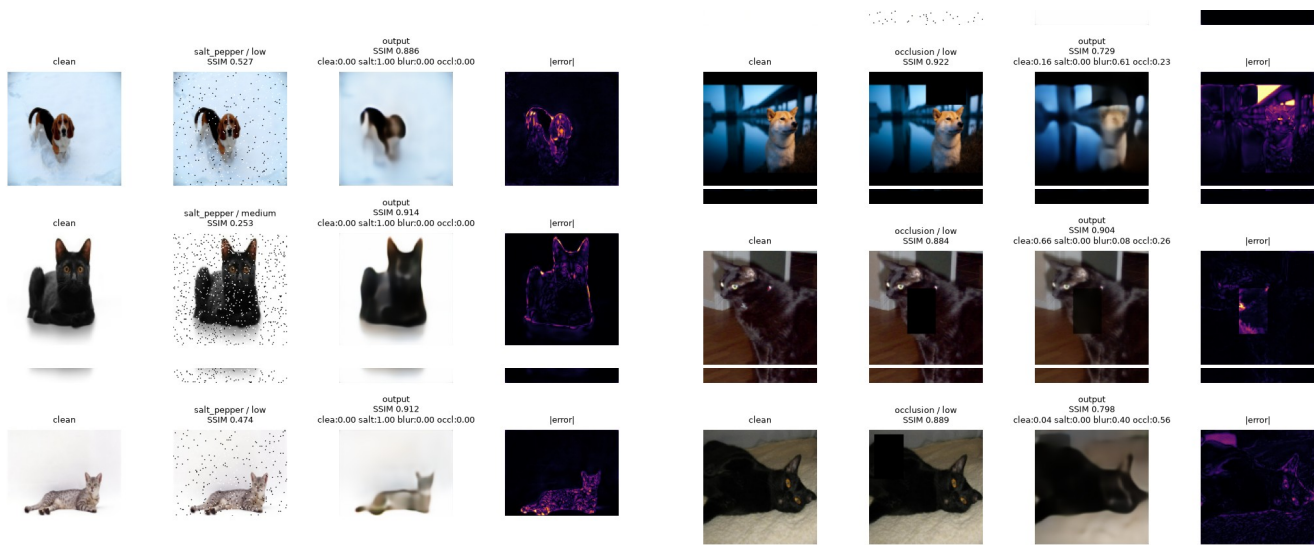


Fig. 12. Task 3: dominant routing (left column: one expert receives weight 1.00) versus distributed routing (right column: weights spread over several branches, shown above each output as clean/salt/blur/occlusion).

free of errors (99.6% accuracy) and its main benefit is the identity bypass for clean images; soft routing reaches the same accuracy while hedging on ambiguous low-severity occlusions, which gives the best occlusion SSIM. The decisive factor for restoration quality, however, is architectural: the required compressed bottleneck caps all three restoration systems at about 23 dB, and one narrow skip connection removes most of that gap. For sketches, a learned style embedding in both networks produces clearly different styles from the same photograph. The deployment is reproducible from a fresh clone with one command and is tested automatically.

**Links.** Repository: [https://github.com/Abubakar7-dotcom/Gen\\_ai\\_assignment1](https://github.com/Abubakar7-dotcom/Gen_ai_assignment1). Trained models: [https://github.com/Abubakar7-dotcom/Gen\\_ai\\_assignment1/releases/tag/models-v1](https://github.com/Abubakar7-dotcom/Gen_ai_assignment1/releases/tag/models-v1). Experiment tracking: <https://wandb.ai/abubaknaveed-national-university-of-computer-and-emergi/genai-a1>. Demonstration video: [\[YouTube link of the demo video\]](#).

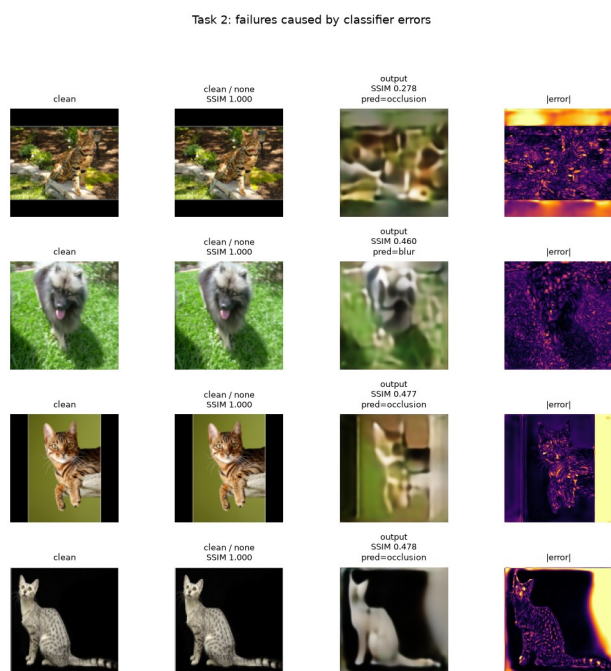


Fig. 13. Task 2 failures caused by classifier errors: clean images with real black borders are classified as occluded (“pred”) and blurred by the occlusion expert.

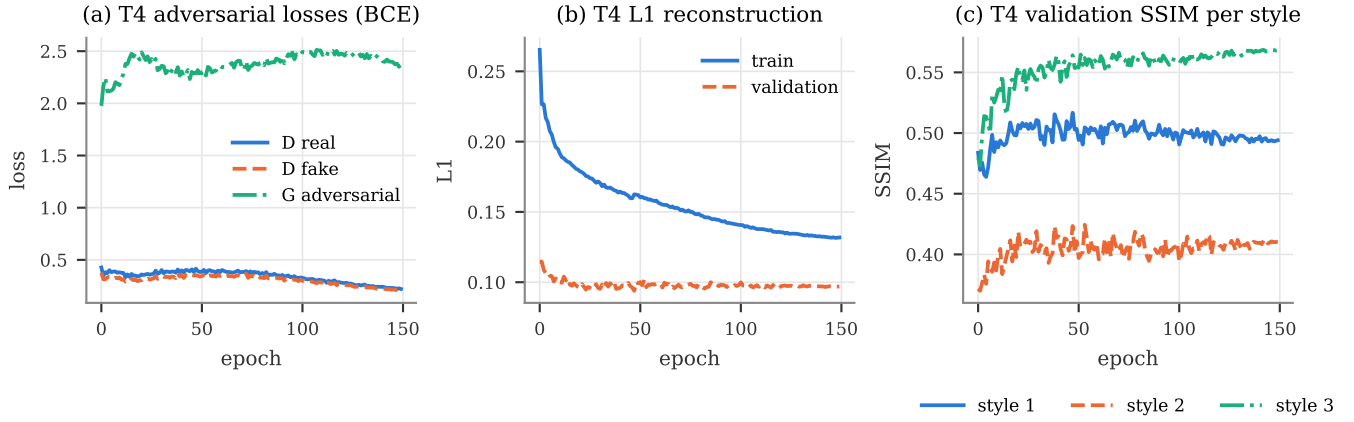


Fig. 14. Task 4 losses logged separately: discriminator real/fake and generator adversarial losses, generator  $\ell_1$  on training versus validation, and validation SSIM per style.

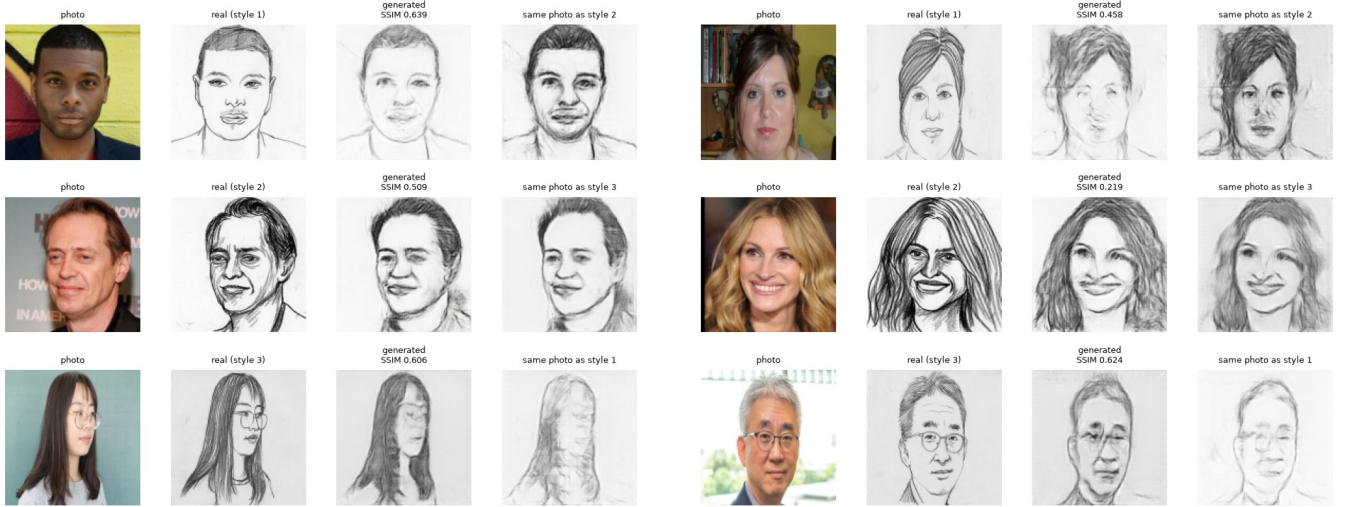


Fig. 15. Task 4 test examples, two per style (rows: style 1, 2, 3). Columns per example: photo, real sketch, generated sketch in the true style, and the same photo generated in another style.

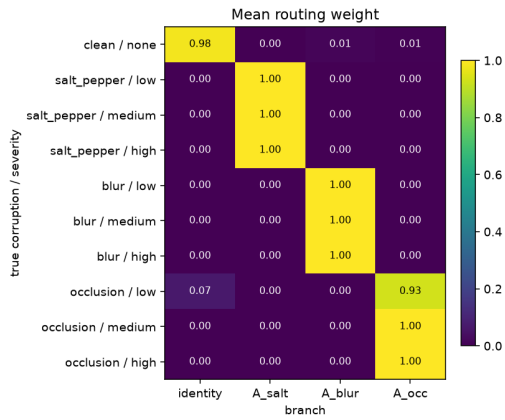


Fig. 16. Task 3 routing heatmap: mean gate weight per branch for every true corruption and severity (test set).

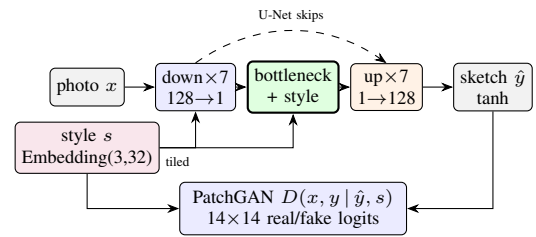


Fig. 17. Task 4 conditional GAN. The learned style embedding enters the generator at the input and at the bottleneck, and the discriminator receives photo, (real or generated) sketch and the tiled embedding.

[Google Stitch design screenshots: save them as docs/stitch/01\_universal.png, 02\_hard.png, 03\_moe.png, 04\_sketch.png and run `python -m scripts.report_figures`]

Fig. 18. Original Google Stitch design of the four workspaces (evidence of the design step).

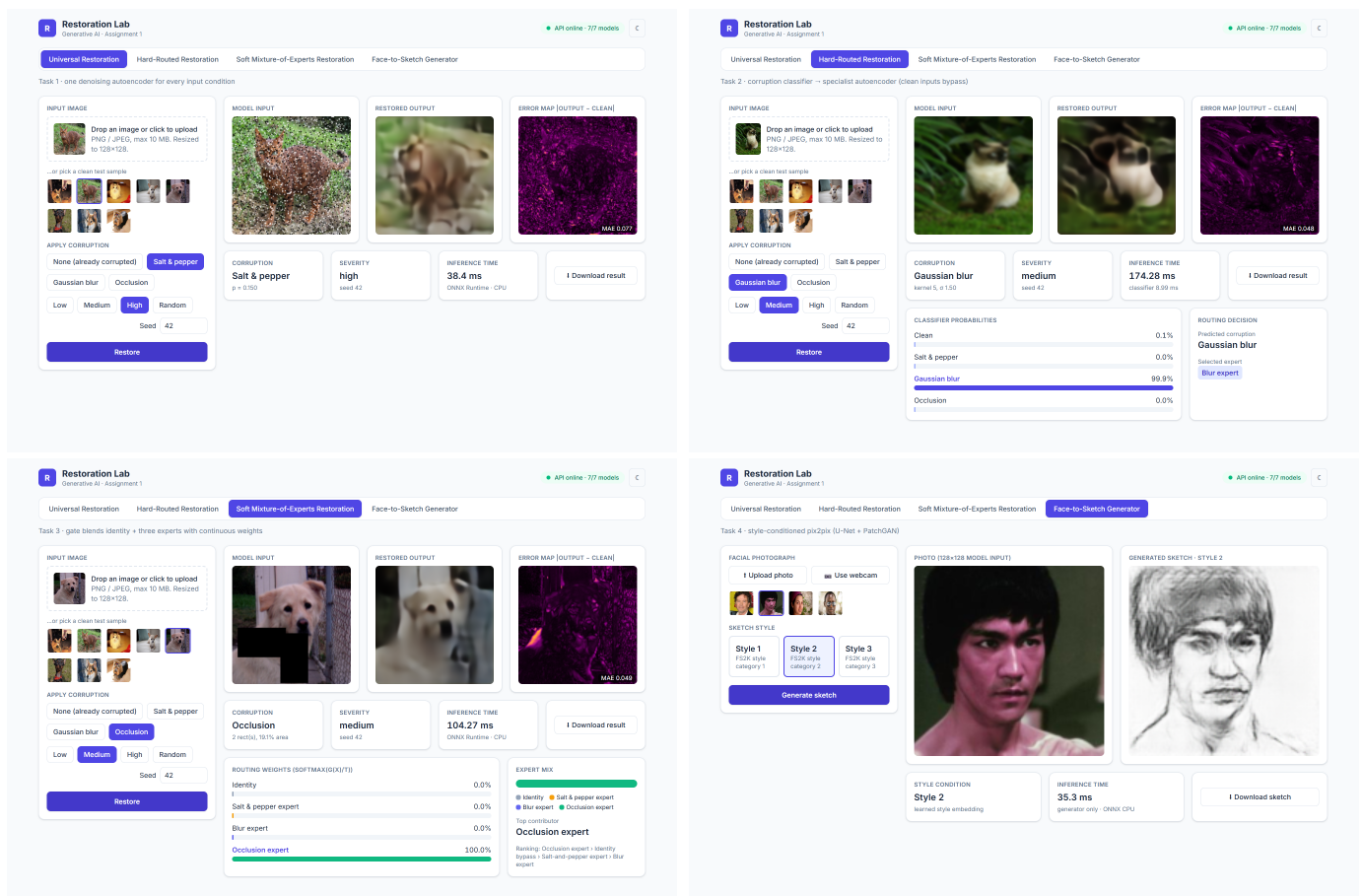


Fig. 19. The deployed application with the trained models. Top left: Universal Restoration (salt-and-pepper applied at runtime, input / output / error map, corruption settings, inference time, download). Top right: Hard-Routed Restoration of an *uploaded* photograph that is not part of the dataset samples (classifier probabilities, predicted corruption, selected expert). Bottom left: Soft Mixture-of-Experts Restoration (all four routing weights, expert-mix bar, top contributor). Bottom right: Face-to-Sketch Generator (style selection, side-by-side result, download; webcam capture is available through the “Use webcam” button).

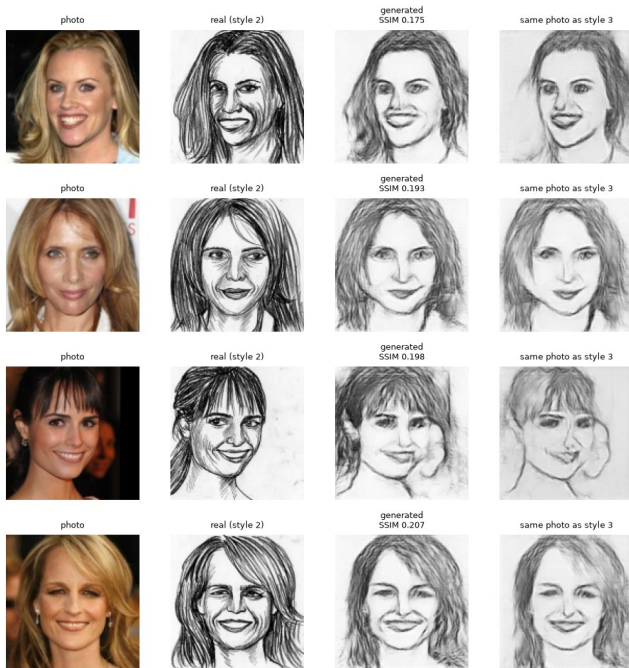


Fig. 20. Task 4 failure cases (lowest SSIM): style-2 hatching on long hair.

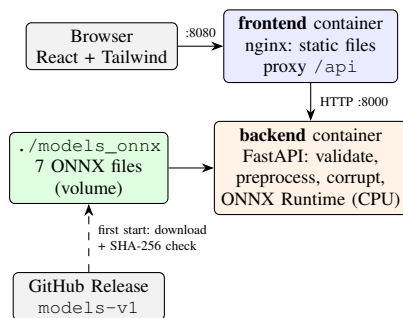


Fig. 21. Deployment: one docker compose up --build starts both containers; the backend downloads and verifies the models on its first start.

## REFERENCES

- [1] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proc. ICML*, 2008, pp. 1096–1103.
- [2] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *J. Mach. Learn. Res.*, vol. 11, pp. 3371–3408, 2010.
- [3] X.-J. Mao, C. Shen, and Y.-B. Yang, "Image restoration using very deep convolutional encoder-decoder networks with symmetric skip connections," in *Proc. NIPS*, 2016, pp. 2802–2810.
- [4] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017.
- [5] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [6] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, "Loss functions for image restoration with neural networks," *IEEE Trans. Comput. Imaging*, vol. 3, no. 1, pp. 47–57, 2017.
- [7] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Comput.*, vol. 3, no. 1, pp. 79–87, 1991.
- [8] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.
- [9] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.
- [10] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Proc. NIPS*, 2014, pp. 2672–2680.
- [11] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv:1411.1784, 2014.
- [12] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. CVPR*, 2017, pp. 1125–1134.
- [13] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.
- [14] D. Ulyanov, A. Vedaldi, and V. Lempitsky, "Instance normalization: The missing ingredient for fast stylization," arXiv:1607.08022, 2016.
- [15] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. Van Gool, "Facial-sketch synthesis: A new challenge," *Mach. Intell. Res.*, vol. 19, no. 4, pp. 257–287, 2022.
- [16] X. Wang and X. Tang, "Face photo-sketch synthesis and recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 31, no. 11, pp. 1955–1967, 2009.
- [17] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in *Proc. CVPR*, 2012, pp. 3498–3505.
- [18] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD*, 2019, pp. 2623–2631.
- [19] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyperparameter optimization," in *Proc. NIPS*, 2011, pp. 2546–2554.
- [20] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.
- [21] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [22] A. Odena, V. Dumoulin, and C. Olah, "Deconvolution and checkerboard artifacts," *Distill*, 2016. [Online]. Available: <https://distill.pub/2016/deconv-checkerboard/>
- [23] L. Biewald, "Experiment tracking with Weights and Biases," 2020. [Online]. Available: <https://www.wandb.com/>
- [24] ONNX Runtime developers, "ONNX Runtime," 2021. [Online]. Available: <https://onnxruntime.ai/>

## APPENDIX A USE OF AI TOOLS

AI assistants were used as permitted by the brief. Table IX lists the tools, what they were used for and how their out-

put was checked; the full log is `docs/ai_use.md` in the repository.

TABLE IX  
AI-TOOL USE AND VERIFICATION

| Tool                 | Used for                                                                        | How the output was verified or corrected                                                                                                                                                                                           |
|----------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude (Cowork)      | Breaking down the brief, project plan, repository scaffold                      | Checked requirement by requirement against the assignment PDF                                                                                                                                                                      |
| Claude               | Corruption module, data pipeline, manifests                                     | Unit tests (parameter ranges, occlusion union area, determinism from seed); visual sanity grid                                                                                                                                     |
| Claude Code          | Models, training engines, Optuna studies, evaluation, ONNX export               | CPU smoke tests on synthetic data; ONNX parity check ( $< 10^{-4}$ ); metrics cross-checked against per-item CSV files                                                                                                             |
| Claude Code (GPU PC) | Data download, running the studies and training, efficiency fixes, ablations    | Each change measured before adoption ( <code>cudnn.benchmark</code> , data-loader workers); mirror data asserted equal to the official image lists; failures (memory, power loss) diagnosed from logs and resumed from checkpoints |
| Claude Code          | FastAPI backend, React frontend, Docker files, model auto-download, CI workflow | End-to-end test of all endpoints through nginx; browser test of every workspace with an unseen uploaded image; GitHub Actions fresh-clone <code>docker compose</code> test                                                         |
| Claude Code          | Figures and first draft of this report                                          | Every number taken from the result files in <code>results/</code> ; text reviewed and edited by the author                                                                                                                         |